

File Edit Selection View Go Run Terminal Help

temp.cpp

```
1 //Linear Search
2 #include <bits/stdc++.h>
3 using namespace std;
4
5 int main() {
6     int n; cin >> n;
7     vector<int> a(n);
8     for(int& x : a) cin >> x;
9     int susID; cin >> susID;
10
11     bool found = false;
12     for(int& x : a) {
13         if(x == susID) {
14             found = true;
15             break;
16         }
17     }
18
19     if(found) cout << "Fraud Transaction Found";
20     else cout << "Transaction Not Found";
21 }
```

input.txt

```
1 5
2 1201 1305 1450 1408 1502
3 1450
```

output.txt

```
1 Transaction Not Found
```

Ln 3, Col 5 Spaces: 4 UTF-8 CRLF {} Plain Text Go Live

ENG IN 57% 15:37 21-06-2026

LOGIC: Linear Search

1. We will take the relevant inputs.
2. We will traverse through all the elements and compare with the target at each iteration.
3. If we find the element, we can break early and save some time as we only have to return true or false.
4. else we print, not found.

FileEditSelectionViewGoRunTerminalHelp

temp.cpp

```
temp.cpp > main()
1 //Binary Search
2 #include <bits/stdc++.h>
3 using namespace std;
4
5 int main() {
6     int n; cin >> n;
7     vector<int> a(n);
8     for(int& x : a) cin >> x;
9     int logID; cin >> logID;
10
11     bool found = false;
12     int low = 0;
13     int high = n - 1;
14
15     while(low <= high) {
16         int mid = (low + high) / 2;
17         if(a[mid] == logID) {
18             found = true;
19             break;
20         }
21         else if(a[mid] > logID) high = mid - 1;
22         else low = mid + 1;
23     }
24
25     if(found) cout << "Log Found";
26     else cout << "Log Not Found";
27 }
```

input.txt

```
1 7
2 1001 1005 1010 1015 1020 1030 1040
3 1015
```

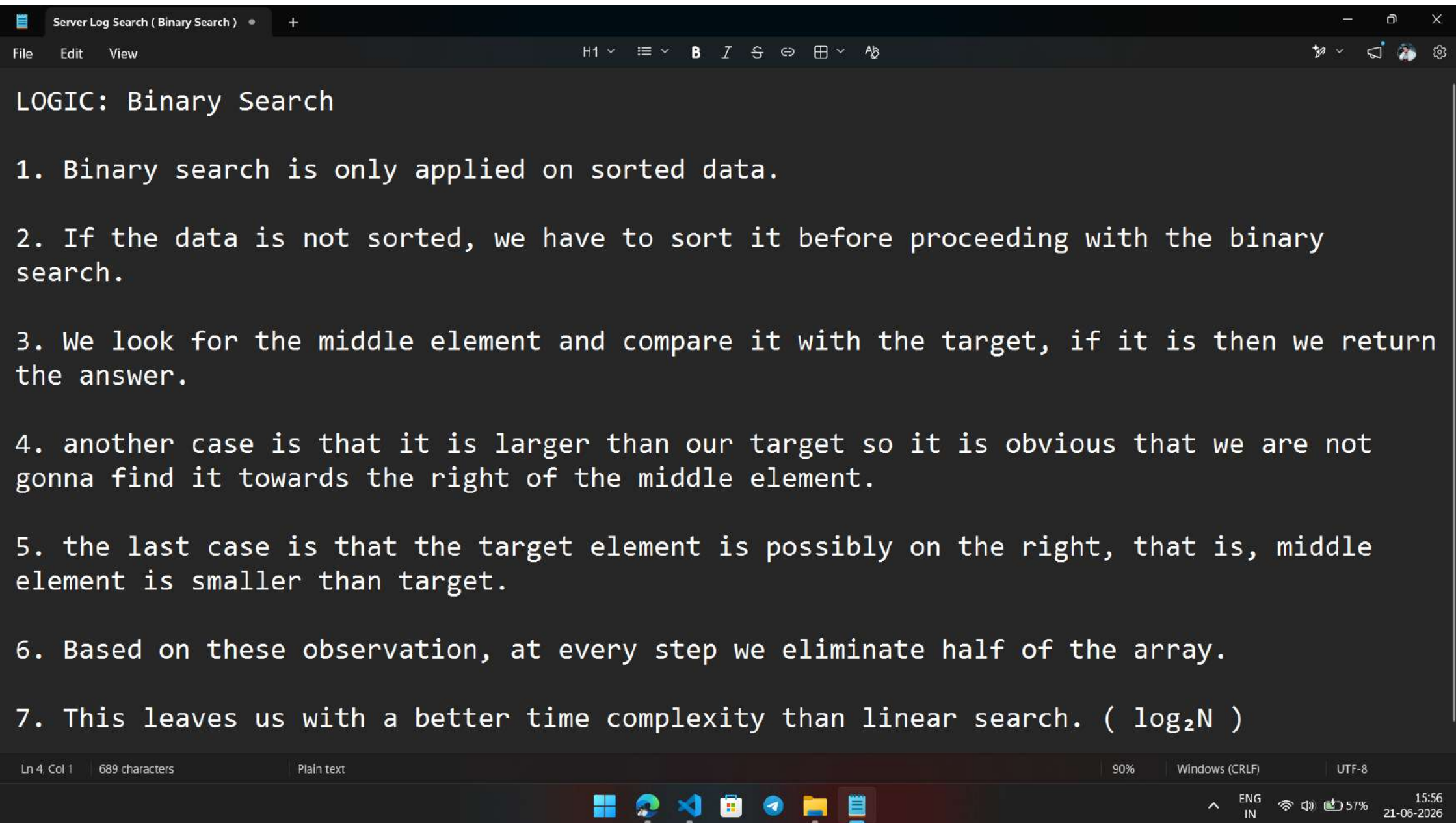
output.txt

```
1 Log Found
```

Ln 16, Col 36 Spaces: 4 UTF-8 CRLF { } C++ Go Live Win32

15:48 21-06-2026

ENG IN 57%



```
Server Log Search ( Binary Search )
File Edit View
H1
1. Binary search is only applied on sorted data.
2. If the data is not sorted, we have to sort it before proceeding with the binary search.
3. We look for the middle element and compare it with the target, if it is then we return the answer.
4. another case is that it is larger than our target so it is obvious that we are not gonna find it towards the right of the middle element.
5. the last case is that the target element is possibly on the right, that is, middle element is smaller than target.
6. Based on these observation, at every step we eliminate half of the array.
7. This leaves us with a better time complexity than linear search. ( log2N )
Ln 4, Col 1 | 689 characters | Plain text | 90% | Windows (CRLF) | UTF-8
15:56 21-06-2026
```

FileEditSelectionViewGoRunTerminalHelp

C++

temp.cpp X

temp.cpp > main()
1 // Bubble Sort
2 #include <bits/stdc++.h>
3 using namespace std;
4
5 int main() {
6 int n; cin >> n;
7 vector<int> a(n);
8 for(int& x : a) cin >> x;
9
10 for(int i = 0; i < n - 1; i++) {
11 for(int j = 0; j < n - 1 - i; j++) {
12 if(a[j] > a[j + 1]) swap(a[j], a[j + 1]);
13 }
14 }
15
16 for(int& x : a) cout << x << " ";
17 }

input.txt X

input.txt
1 5
2 45000 32000 55000 28000 60000

output.txt X

output.txt
1 28000 32000 45000 55000 60000

Ln 17, Col 2 Spaces: 4 UTF-8 CRLF {} C++ Go Live Win32

ENG IN 16:04 21-06-2026

LOGIC: Bubble Sort

1. We use two loops as we have to iterate the whole array for each and every element.
2. we compare the adjacent elements and if they are not in ascending, we swap them
3. this makes the largest element bubble up to the end of the array and it takes it's correct place, hence the name, Bubble sort.
4. after each iteration, the largest element takes it's correct place so we can trim the array space by 1 after every iteration.
5. We make sure that the array runs till $n-2$ as we will compare $(n-2)$ th element with $(n-1)$ th element. Any more iteration than this will cause out of bounds error.

FileEditSelectionViewGoRunTerminalHelp

temp.cpp

temp.cpp > main()

```
1 // Insertion Sort
2 #include <bits/stdc++.h>
3 using namespace std;
4
5 int main() {
6     int n; cin >> n;
7     vector<int> a(n);
8     for(int& x : a) cin >> x;
9
10    for (int i = 1; i < n; ++i) {
11        int elem = a[i];
12        int j = i - 1;
13        while (j >= 0 && a[j] > elem) {
14            a[j + 1] = a[j];
15            j = j - 1;
16        }
17        a[j + 1] = elem;
18    }
19
20    for(int& x : a) cout << x << " ";
21 }
```

input.txt

```
1 5
2 250 120 320 180 100
```

output.txt

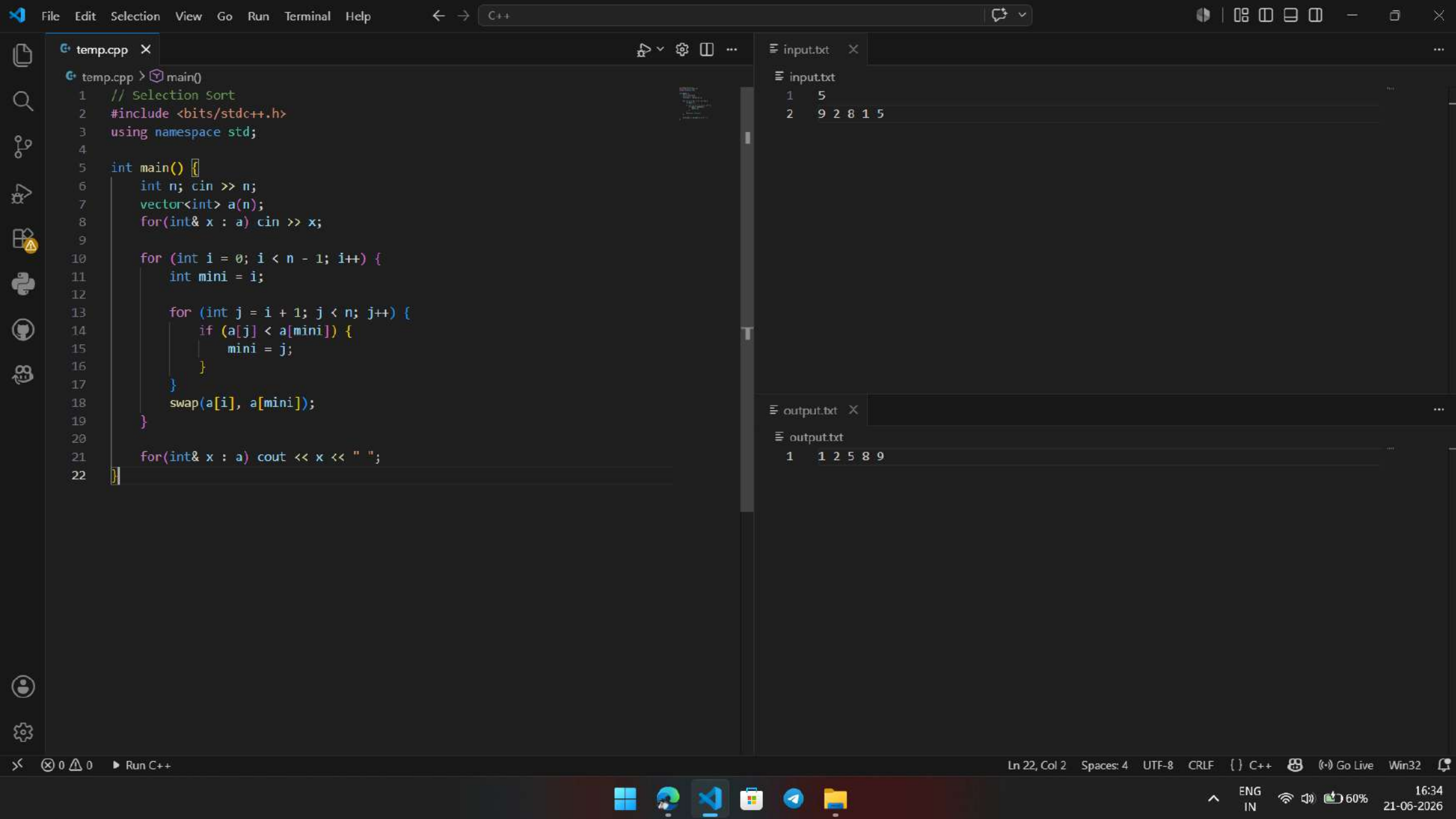
```
1 100 120 180 250 320
```

Ln 21, Col 2 Spaces: 4 UTF-8 CRLF { } C++ Go Live Win32

ENG IN 59% 16:14 21-06-2026

LOGIC: Insertion Sort

1. We assume the first element to be sorted and start iterating from the second element.
2. We create a copy of the i 'th element for later use.
3. We do $j = i - 1$, as we have to start comparing from here till we reach the starting of the array or the element(elem) reaches it's correct place.
4. We go backwards with $j--$ and compare if it's greater, which it is not supposed to be, we insert this value to the front and repeat until the condition "`while(j >= 0 && a[j] > key)`" remains true.
5. After the while loop breaks, it is 100% sure that, that particular place before the last $j--$ was the correct place for our i 'th element.
6. So, we insert the i 'th element at $(j+1)$ 'th index.



LOGIC: Selection Sort

1. We consider the i 'th element to be the smallest.
2. We start the 2nd loop with $i+1$ and go till end and find the real smallest element.
3. after successfully finding the real smallest element, we swap it with the i 'th element.
4. This way, after every iteration, the i 'th index occupies the next larger element and the array becomes sorted.